Introduction to PostgreSQL & SQL Basics

Section 1

What is a Database & Why We Use It

What is a Database?

A database is an organized collection of structured data stored electronically, designed for efficient access, management, and updating.

📦 Store

Holds large amounts of information in an organized way — text, numbers, images, files.

🔍 Retrieve

Quickly find specific data using queries — even across millions of records instantly.

🔒 Protect

Control who can read, edit, or delete data with roles, permissions, and backups.

Key Features of a Database

🗂️ Data Organization

Data is stored in structured formats — tables, rows, and columns — making it easy to find and manage.

🔄 Data Integrity

Constraints (PRIMARY KEY, NOT NULL, UNIQUE) prevent invalid or duplicate data from being stored.

⚡ Fast Access

Indexes and query optimizers allow retrieval of millions of records in milliseconds.

🔁 Concurrency

Multiple users can read/write simultaneously without data corruption — handled by transactions.

🔐 Security & Access Control

Role-based permissions ensure only authorized users can view or modify sensitive data.

💾 Backup & Recovery

Databases support point-in-time recovery, replication, and automatic backups to prevent data loss.

Methods for Storing Data

📄 Flat Files

CSV, TXT files. Simple but no relationships, no indexing, hard to query large volumes.

📊 Spreadsheets

Excel/Google Sheets. Good for small datasets but not scalable for applications.

🗄️ Relational DB

MySQL, PostgreSQL. Tables with relationships, SQL queries, ACID transactions.

📜 NoSQL DB

MongoDB, Redis. Document, key-value, graph stores — flexible schema, high scale.

☁️ Cloud Storage

S3, GCS. Object storage for files, images, videos — not for structured queries.

🧠 In-Memory DB

Redis, Memcached. Extremely fast (RAM-based) — great for caching, sessions.

Databases in Real Life — Real Examples

🛒 Amazon / Shopify

Product catalog, orders, inventory, customer accounts, payments

🎬 Netflix / YouTube

User profiles, watch history, recommendations, video metadata

🏦 Banks / PayPal

Account balances, transactions, fraud detection, audit logs

🏥 Hospitals / EMR

Patient records, prescriptions, lab results, appointments

📱 WhatsApp / Telegram

Messages, contacts, group memberships, delivery receipts

✈️ Airlines / Booking.com

Flight schedules, seat reservations, passenger manifests, pricing

Section 2

Database Management System (DBMS)

What is a DBMS? — Definition

A Database Management System (DBMS) is software that enables users and applications to define, create, maintain, and control access to a database.

- **Application Layer**

  Your app, website, or script sends SQL queries

- **DBMS Engine**

  Parses queries, optimizes execution plans, enforces rules

- **Storage Layer**

  Data files, indexes, logs stored on disk or SSD

Why is a DBMS Important?

Data Security

Controls who can see, edit, or delete data. Passwords, roles, and audit logs are built in.

Data Consistency

Transactions ensure all-or-nothing operations. A bank transfer either fully completes or fully rolls back.

Performance

Query optimizers, indexes, and caching let you query billions of rows in seconds.

Multi-User Access

Thousands of users can query simultaneously without overwriting each other's data.

Backup & Recovery

Automated backups, WAL logs, and point-in-time recovery protect against disasters.

Data Independence

Application code doesn't need to know how data is physically stored or organized on disk.

Popular DBMS Software

PostgreSQL

Relational (Open Source)

Advanced features, ACID, extensions

MySQL

Relational (Open Source)

Fast, widely used in web apps (LAMP stack)

SQL Server

Relational (Microsoft)

Enterprise, deep .NET & Azure integration

Oracle DB

Relational (Commercial)

Industry giant — banking, ERP, very expensive

MongoDB

Document (NoSQL)

JSON-like documents, flexible schema, Node.js

Redis

Key-Value (In-Memory)

Sub-millisecond speed — caching, sessions

SQLite

Embedded Relational

Zero-config, used in mobile apps & browsers

Cassandra

Wide-Column (NoSQL)

Massive scale, high availability — Facebook

Section 3

All Types of Databases

All Types of Databases

Relational (SQL)

e.g. PostgreSQL, MySQL, SQLite

Data in tables with rows & columns. Use SQL to query. Best for structured, consistent data.

Document

e.g. MongoDB, CouchDB

Store JSON-like documents. Flexible schema — each document can have different fields.

Key-Value

e.g. Redis, DynamoDB

Simplest model — key maps to a value. Lightning fast — great for caching & sessions.

Wide-Column

e.g. Cassandra, HBase

Rows with dynamic columns. Used for analytics and massive write-heavy workloads.

Graph

e.g. Neo4j, Amazon Neptune

Nodes and edges. Perfect for relationships — social networks, fraud detection, recommendations.

Time-Series

e.g. InfluxDB, TimescaleDB

Optimized for data with timestamps — IoT sensors, stock prices, monitoring metrics.

Section 4

SQL vs NoSQL — Importance & Comparison

Why SQL is Important

SQL (Structured Query Language) is the universal language for working with relational databases — used by virtually every major software system.

01

Universal Standard

ISO standard since 1987 — the same SQL concepts work across PostgreSQL, MySQL, Oracle, SQL Server, and more.

02

Massive Industry Adoption

SQL is among the top skills in data engineering, backend development, and data analysis job listings worldwide.

03

Powerful Data Retrieval

JOINs, aggregations, window functions, and subqueries let you answer complex business questions in one statement.

04

ACID Transactions

SQL databases guarantee Atomicity, Consistency, Isolation, Durability — critical for financial and medical systems.

Why NoSQL is Important

NoSQL databases emerged to handle the massive scale, flexibility, and speed requirements that traditional relational databases struggled with.

01

Flexible Schema

No fixed table structure — add new fields at any time without altering the whole table. Great for agile development.

02

Horizontal Scaling

NoSQL databases can scale across thousands of servers (sharding) — handling petabytes of data like Netflix and Twitter.

03

High Write Throughput

Cassandra and DynamoDB can handle millions of writes per second — critical for real-time logging and IoT.

04

Diverse Data Models

Documents, graphs, key-value — choose the model that fits your data, not the other way around.

SQL vs NoSQL — Side-by-Side Comparison

Feature	SQL (Relational)	NoSQL
Schema	Fixed — defined in advance	Dynamic — flexible, can change
Query Language	SQL (standardized)	Varies by DB (no universal standard)
Scaling	Vertical (bigger server)	Horizontal (more servers)
Data Model	Tables, rows, columns	Documents, key-value, graph, etc.
ACID	Full ACID guarantees	Often eventual consistency (BASE)
Best For	Financial, ERP, reporting	Big data, real-time, flexible apps
Examples	PostgreSQL, MySQL, Oracle	MongoDB, Redis, Cassandra, DynamoDB

Section 5

Relational Database Concepts

Tables, Rows, Columns & Keys — Core Concepts

customer_id	name	email	city
1	Ahmed Ali	ahmed@mail.com	Cairo
2	Sara Hassan	sara@mail.com	Dubai
3	John Smith	john@mail.com	London

← Column (field/attribute)

← Primary Key (unique per row)

← Row / Record / Tuple

Table:

A named collection of related data organized in rows and columns (like a spreadsheet). E.g. customers, orders, products.

Row:

A single record / entry in the table. E.g. one customer. Also called a 'tuple' or 'record'.

Column:

An attribute/field shared by all rows. E.g. 'email' column exists for every customer.

Primary Key:

A column (or combination) that uniquely identifies each row. Cannot be NULL or duplicate. E.g. customer_id.

Foreign Keys & Relationships

customers

customer_id (PK)	name
1	Ahmed Ali
2	Sara Hassan

FK references PK →

orders

order_id (PK)	customer_id (FK)	total
101	1	$250
102	2	$180
103	1	$390

Foreign Key (FK):

A column in one table that references the Primary Key of another table, creating a relationship between tables.

One-to-Many:

One customer can have MANY orders. The FK (customer_id) in orders points to the PK in customers.

Referential Integrity:

The DBMS ensures you cannot add an order for customer_id=99 if customer 99 does not exist.

Section 6

Introduction to PostgreSQL — History, Features & Advantages

PostgreSQL — History & Background

1986

POSTGRES project

Started at UC Berkeley by Prof. Michael Stonebraker as a research project.

1994

SQL Added

Developers added SQL interpreter — renamed to Postgres95.

1996

PostgreSQL Born

Name changed to PostgreSQL. First open-source public release.

2010s

Rise to Prominence

JSON support, extensions, logical replication — became enterprise-grade.

2025

PostgreSQL 17

Latest major version with incremental backup, performance improvements, new SQL features.

PostgreSQL — Key Features & Advantages

🏆 ACID Compliant

Full transactional safety — Atomicity, Consistency, Isolation, Durability guaranteed on every write.

📦 Rich Data Types

INTEGER, TEXT, JSONB, ARRAY, UUID, HSTORE, geometric types, custom types — far beyond standard SQL.

🔌 Extensible

PostGIS (geospatial), pgvector (AI embeddings), TimescaleDB (time-series) — PostgreSQL is a platform.

🔍 Advanced Indexing

B-Tree, GiST, GIN, BRIN, partial indexes, expression indexes — highly tunable performance.

🔄 Replication

Built-in streaming replication, logical replication, and support for multi-master via extensions.

📜 Full SQL Standard

Window functions, CTEs, recursive queries, lateral joins — one of the most standards-compliant databases.

🆓 Open Source

Free forever under the PostgreSQL License. No per-core licensing costs unlike Oracle or SQL Server.

🌍 Community & Ecosystem

25+ years of development, massive community, extensive documentation, AWS/Azure/GCP native support.

Section 7

PostgreSQL vs MySQL vs SQL Server

PostgreSQL vs MySQL vs SQL Server — Comparison

Feature	PostgreSQL	MySQL	SQL Server
License	Open Source (free)	Open Source / Commercial	Commercial (Microsoft)
ACID	Full ACID	Full ACID (InnoDB)	Full ACID
JSON Support	JSONB (powerful)	Basic JSON	JSON support
Performance	Complex queries	Simple reads (fast)	Enterprise workloads
Extensions	Massive ecosystem	Limited	T-SQL extensions
Replication	Streaming + logical	Master-slave, group	Always On, AG
Best For	Complex apps, analytics	Web apps (LAMP/LEMP)	Microsoft/.NET ecosystem
Used By	Instagram, Apple, Skype	Facebook, Twitter, Airbnb	Banks, large enterprises

Section 8

Installing PostgreSQL & pgAdmin

⌨ Installing PostgreSQL on Windows — Step by Step

01

Download Installer

Go to postgresql.org/download/windows → click 'Download the installer' (EnterpriseDB)

02

Run the Setup Wizard

Choose components: PostgreSQL Server ✓ pgAdmin 4 ✓ Command Line Tools ✓ Stack Builder (optional)

03

Set Password

Set a password for the superuser 'postgres'. Remember this — you need it to connect!

04

Set Port

Default port is 5432. Keep it unless you have a conflict. Click Next → Next → Finish.

05

Verify Installation

Open Command Prompt: psql -U postgres -c "SELECT version();" → should show PostgreSQL version

Installing PostgreSQL on macOS – Step by Step

Method 1: Using Homebrew (Recommended)

```
# Install Homebrew if not installed:
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install
/HEAD/install.sh)"

# Install PostgreSQL:
brew install postgresql@16

# Start PostgreSQL service:
brew services start postgresql@16

# Connect to verify:
psql postgres
```

Method 2: Postgres.app (GUI way)

```
1. Go to  postgresapp.com
2. Download Postgres.app
3. Move to /Applications
4. Open app → click Initialize
5. Click on a database to open psql
6. Add to PATH (shown in app):
   export PATH=$PATH:/Applications/
   Postgres.app/Contents/Versions/
   latest/bin
```

Using pgAdmin 4 as a GUI

Browser Panel (Left)

Tree view of Servers → Databases → Schemas → Tables. Expand to explore everything visually.

Query Tool

Open with: right-click Database → Query Tool. Write SQL, execute with F5, see results below.

Dashboard

Server-level metrics: active connections, TPS, disk I/O, locks. Great for monitoring performance.

How to Connect a Server in pgAdmin 4:

1. Open pgAdmin 4 → click Add New Server (or right-click Servers → Register → Server)
2. General Tab: Name = MyLocalServer (any name you choose)
3. Connection Tab: Host = localhost | Port = 5432 | Username = postgres | Password = (your password)
4. Click Save → expand the server in the browser tree
5. Right-click a server → Create → Database Name=MyLocalDB (any name you choose)

Connecting via psql Command Line

Basic Syntax:

```
psql -U <username> -d <database_name> -h <host> -p <port>
```

Common Connection Examples:

```
# Connect as postgres (default)
psql -U postgres
```

```
# Connect to specific database
psql -U postgres -d myapp
```

```
# Connect to remote server
psql -U admin -d production -h 10.0.0.5 -p 5432
```

```
# Run a single query and exit
psql -U postgres -d myapp -c "SELECT COUNT(*) FROM users;"
```

Section 9

SQL Data Types in PostgreSQL

PostgreSQL Data Types — Complete Reference

Numeric

INTEGER / INT

Whole numbers: -2B to +2B. e.g. age, count, quantity

BIGINT

Large whole numbers: ±9 quintillion. e.g. user IDs at scale

SERIAL

Auto-incrementing integer. Shorthand for sequence + NOT NULL

NUMERIC(p,s)

Exact decimal — p digits total, s after point. For money!

FLOAT / REAL

Approximate decimal. Fast but can have rounding errors

Text

VARCHAR(n)

Variable text, max n characters. e.g. VARCHAR(100) for name

TEXT

Unlimited length text. No performance penalty vs VARCHAR in PG

CHAR(n)

Fixed-length text — pads with spaces. Rarely used today

Date & Time

DATE

Calendar date: '2024-01-15'. No time component.

TIMESTAMP

Date + time: '2024-01-15 14:30:00'. No timezone.

TIMESTAMPTZ

Timestamp WITH timezone. Stored as UTC, displayed in local TZ

Other

BOOLEAN

TRUE or FALSE (also accepts 't', 'f', '1', '0')

JSON / JSONB

Semi-structured data. JSONB is binary — faster to query

UUID

Universally unique ID: '550e8400-e29b-41d4-a716-...'

⌨ Data Types — Examples in Practice

```
-- Using correct data types in a real table:
CREATE TABLE employees (
    emp_id      SERIAL          PRIMARY KEY,       -- auto-increment integer
    full_name   VARCHAR(150)    NOT NULL,           -- max 150 chars
    bio         TEXT,                               -- unlimited text
    salary      NUMERIC(10, 2)  NOT NULL,           -- e.g. 85000.00
    is_active   BOOLEAN         DEFAULT TRUE,       -- true/false
    hired_at    DATE            DEFAULT CURRENT_DATE,
    last_login  TIMESTAMPTZ,                        -- with timezone
    profile     JSONB                               -- flexible metadata
);
```

✅ **Correct Type Selection Tips:**

💰 Money amounts → NUMERIC(10,2), never FLOAT | 🔑 Auto IDs → SERIAL or BIGSERIAL | 🕐 Dates with TZ → TIMESTAMPTZ

Section 10

DDL — CREATE, ALTER, DROP

DDL: CREATE DATABASE & CREATE TABLE

CREATE DATABASE

Creates a new, empty database. You need superuser or CREATEDB privilege to run this.

💡 *One database per project is typical. Databases are isolated namespaces.*

CREATE TABLE

Defines a new table with column names, data types, and constraints. This is where your data will live.

💡 *Choose column types carefully — harder to change later at scale.*

See the next slide for full code examples →

⌨ CREATE DATABASE & CREATE TABLE — Examples

```
-- Example 1: Create a new database
CREATE DATABASE ecommerce;

-- Example 2: Connect to it (in psql)
\c ecommerce

-- Example 3: Create a simple table
CREATE TABLE products (
    product_id   SERIAL          PRIMARY KEY,
    name         VARCHAR(200)    NOT NULL,
    price        NUMERIC(10, 2)  NOT NULL  CHECK (price >= 0),
    stock        INTEGER         DEFAULT 0,
    created_at   TIMESTAMP       DEFAULT NOW()
);

-- Example 4: Create a related orders table
CREATE TABLE orders (
    order_id     SERIAL      PRIMARY KEY,
    product_id   INTEGER     REFERENCES products(product_id),
    quantity     INTEGER     NOT NULL  CHECK (quantity > 0),
    order_date   DATE        DEFAULT CURRENT_DATE
);
```

✅ **Result: Two linked tables — products → orders via FOREIGN KEY**

ALTER TABLE — Modify Existing Tables

ALTER TABLE lets you modify an existing table's structure — add columns, rename, change types, add/drop constraints.

```
-- Add a new column:
ALTER TABLE products ADD COLUMN category VARCHAR(100);

-- Add a column with default value:
ALTER TABLE products ADD COLUMN is_active BOOLEAN DEFAULT TRUE;

-- Rename a column:
ALTER TABLE products RENAME COLUMN name TO product_name;

-- Change data type:
ALTER TABLE products ALTER COLUMN stock TYPE BIGINT;

-- Set a NOT NULL constraint:
ALTER TABLE products ALTER COLUMN category SET NOT NULL;

-- Drop a column:
ALTER TABLE products DROP COLUMN is_active;

-- Rename a table:
ALTER TABLE products RENAME TO items;
```

⚠ **Be careful with ALTER on large tables in production — some changes lock the table!**

⌨ DROP TABLE — Delete Tables & Databases

DROP permanently removes a table or database and ALL its data. There is no undo — always backup first!

```
-- Drop a table (fails if it doesn't exist):
DROP TABLE orders;

-- Safe drop (no error if table doesn't exist):
DROP TABLE IF EXISTS orders;

-- Drop table AND all dependent tables (CASCADE):
DROP TABLE products CASCADE;
-- ⚠️ This also drops orders table (references products)!

-- Drop a database:
DROP DATABASE IF EXISTS ecommerce;
-- ⚠️ You cannot drop a database you are connected to!

-- Practical safe pattern before recreating:
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS products;
CREATE TABLE products ( ... );
CREATE TABLE orders ( ... );
```

🔴 **RULE: Never run DROP in production without a recent backup. Double-check table name!**

Section 11

Constraints — Primary Key, Foreign Key, NOT NULL, UNIQUE, CHECK, DEFAULT

Constraints — Theory & Purpose

Constraints are rules applied to columns to enforce data integrity — they prevent invalid data from entering your database.

PRIMARY KEY

Uniquely identifies each row. Cannot be NULL or duplicate. One per table (can be composite).

FOREIGN KEY

Links to a PRIMARY KEY in another table. Enforces referential integrity — can't reference non-existent rows.

NOT NULL

Column must always have a value — empty (NULL) is not allowed. Use for required fields.

UNIQUE

All values in the column must be distinct. Unlike PRIMARY KEY, UNIQUE allows one NULL.

CHECK

Custom condition that every value must satisfy. e.g. CHECK (price > 0), CHECK (age BETWEEN 0 AND 150).

DEFAULT

Value automatically assigned when INSERT doesn't specify this column. e.g. DEFAULT NOW(), DEFAULT TRUE.

⌨ Constraints — Full Code Examples

```
CREATE TABLE users (
    -- PRIMARY KEY: unique, not null, auto-increment
    user_id      SERIAL          PRIMARY KEY,

    -- NOT NULL: username is required
    username     VARCHAR(50)     NOT NULL,

    -- UNIQUE: no two users same email
    email        VARCHAR(200)    NOT NULL  UNIQUE,

    -- CHECK: age must be between 13 and 120
    age          INTEGER         CHECK (age BETWEEN 13 AND 120),

    -- DEFAULT: account is active when created
    is_active    BOOLEAN         DEFAULT TRUE,

    -- DEFAULT: automatically set registration time
    registered   TIMESTAMPTZ     DEFAULT NOW()
);

);
```

✅ ON DELETE CASCADE: if a user is deleted, all their posts are automatically deleted too.

⌨ Constraints — Full Code Examples

```
CREATE TABLE posts (
    post_id     SERIAL      PRIMARY KEY,
    -- FOREIGN KEY: must reference existing user
    user_id     INTEGER     NOT NULL  REFERENCES users(user_id)
                            ON DELETE CASCADE,
    title       TEXT        NOT NULL,
    views       INTEGER     DEFAULT 0  CHECK (views >= 0)
```

✅ ON DELETE CASCADE: if a user is deleted, all their posts are automatically deleted too.

Day 1 Complete! 🎉

- ✅ What is a Database — types, features, real examples
- ✅ DBMS — definition, importance, popular software
- ✅ SQL vs NoSQL — when to use each
- ✅ Relational concepts — tables, rows, columns, keys
- ✅ PostgreSQL — history, features, advantages
- ✅ Installing PostgreSQL & pgAdmin on Windows & Mac
- ✅ Data Types — INTEGER, SERIAL, VARCHAR, TEXT, DATE, JSONB...
- ✅ DDL — CREATE, ALTER, DROP with full examples
- ✅ Constraints — PK, FK, NOT NULL, UNIQUE, CHECK, DEFAULT